BITS Pilani

Pilani | Dubai | Goa | Hyderabad

S1-25_DSECLZG530/SSCLZG599

Natural Language Processing

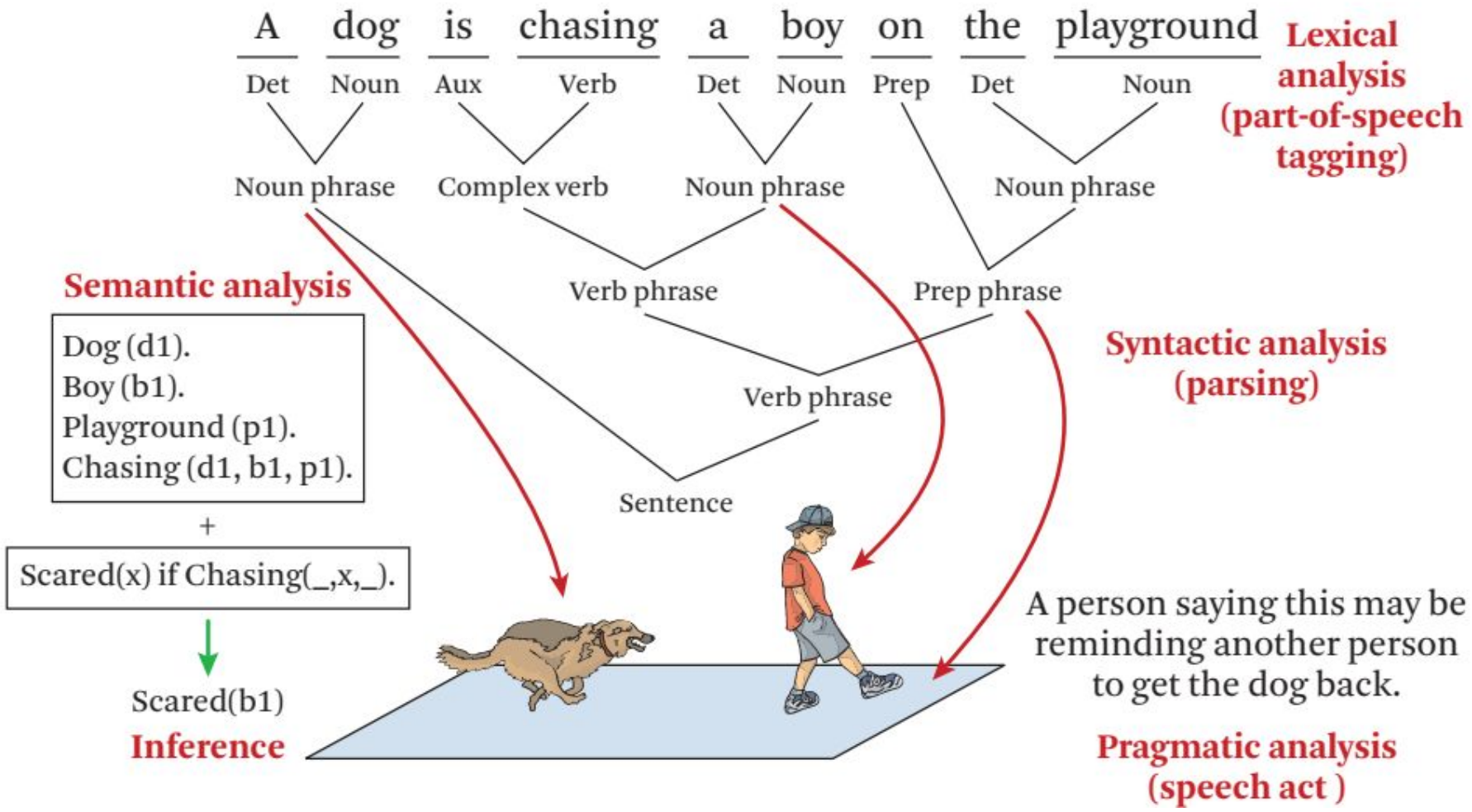
(Lecture #8 – Mid-semester Review)

- *The slides presented here are obtained from the authors of the books and from various other contributors. I hereby acknowledge all the contributors for their material and inputs.*
- *I have added and modified a few slides to suit the requirements of the course.*

Natural Language Processing

- NLP is the branch of computer science focused on developing systems that allow computers to communicate with people using everyday language.
- Also called Computational Linguistics
  - Also concerns how computational methods can aid the understanding of human language
  - The Association for Computational Linguistics (ACL) is a scientific and professional organization for people working on natural language processing

Natural Language Processing



Text Data Management and Analysis: A Practical Introduction to Information Retrieval and Text Mining ChengXiang Zhai, Sean Massung, ACM Books 2016

Turing Test

The Turing Test is a method of inquiry in artificial intelligence for determining whether or not a computer is capable of thinking like a human being

The original Turing Test requires three terminals, each of which is physically separated from the other two. One terminal is operated by a computer, while the other two are operated by humans.

During the test, one of the humans functions as the questioner, the second human and the computer function as respondents.

The questioner interrogates the respondents within a specific subject area, using a specified format and context.

After a preset length of time or number of questions, the questioner is then asked to decide which respondent was human and which was a computer.

Text Normalization

- Every NLP task requires text normalization:
  1. Tokenizing (segmenting) words
  2. Normalizing word formats
  3. Segmenting sentences

- A very simple way to tokenize
  - For languages that use space characters between words
    - Arabic, Cyrillic, Greek, Latin, etc., based writing systems
  - Segment off a token between instances of spaces

Word Normalization

- Putting words/tokens in a standard format
  - U.S.A. or USA
  - uhhuh or uh-huh
  - Fed or fed
  - am, is, be, are
- Case folding
  - Applications like IR: reduce all letters to lower case
    - Since users tend to use lower case
    - Possible exception: upper case in mid-sentence?
      - e.g., ***General Motors***
      - ***Fed*** vs. ***fed***
      - ***SAIL*** vs. ***sail***
  - For sentiment analysis, MT, Information extraction
    - Case is helpful (***US*** versus ***us*** is important)

Another option for text tokenization

Instead of

- white-space segmentation
- single-character segmentation

Use the data to tell us how to tokenize.

Subword tokenization (because tokens can be parts of words as well as whole words)

BPE token learner algorithm

function BYTE-PAIR ENCODING(strings C, number of merges k) **returns** vocab V

$V \leftarrow$ all unique characters in C # initial set of tokens is characters
for $i = 1$ **to** k **do** # merge tokens til k times
  $t_L, t_R \leftarrow$ Most frequent pair of adjacent tokens in C
  $t_{NEW} \leftarrow t_L + t_R$ # make new token by concatenating
  $V \leftarrow V + t_{NEW}$ # update the vocabulary
  Replace each occurrence of t_L, t_R in C with t_{NEW} # and update the corpus
return V

Meaning of a word as a vector

Called an "embedding" because it's embedded into a space

The standard way to represent meaning in NLP

> ***Modern NLP algorithm uses embeddings as the representation of word meaning***

Fine-grained model of meaning for similarity

Consider sentiment analysis:

- With **words**, a feature is a word identity
  - Feature 5: 'The previous word was "terrible"'
  - requires **exact same word** to be in training and test
- With **embeddings**:
  - Feature is a word vector
  - 'The previous word was vector [35,22,17…]
  - Now in the test set we might see a similar vector [34,21,14]
  - We can generalize to **similar but unseen** words!!!

2 kinds of embeddings

tf-idf

- Information Retrieval workhorse!
- A common baseline model
- **Sparse** vectors
- Words are represented by (a simple function of) the **counts** of nearby words

Word2vec

- **Dense** vectors
- Representation is created by training a classifier to **predict** whether a word is likely to appear nearby

Pointwise Mutual Information

Pointwise mutual information:

Do events x and y co-occur more than if they were independent?

$$\text{PMI}(X,Y) = \log_2 \frac{P(x,y)}{P(x)P(y)}$$

PMI between two words: (Church & Hanks 1989)

Do words x and y co-occur more than if they were independent?

$$\text{PMI}(word_1, word_2) = \log_2 \frac{P(word_1, word_2)}{P(word_1)P(word_2)}$$

$$\text{PPMI}(word_1, word_2) = \max\left(\log_2 \frac{P(word_1, word_2)}{P(word_1)P(word_2)}, 0\right)$$

Word2vec: how to learn vectors

Given the set of positive and negative training instances, and an initial set of embedding vectors

The goal of learning is to adjust those word vectors such that we:

- **Maximize** the similarity of the target word, context word pairs (w , c_{pos}) drawn from the positive data
- **Minimize** the similarity of the (w , c_{neg}) pairs drawn from the negative data.

Probabilistic Language Models

Our goal: assign a probability to a sentence

- Machine Translation:
  - P(**high** winds tonite) > P(**large** winds tonite)
- Spell Correction
  - The office is about fifteen **minuets** from my house
    - P(about fifteen **minutes** from) > P(about fifteen **minuets** from)
- Speech Recognition
  - P(I saw a van) >> P(eyes awe of an)
- + Summarization, question-answering, etc., etc.!!

Goal: compute the probability of a sentence or sequence of words:

$$P(W) = P(w_1,w_2,w_3,w_4,w_5 \ldots w_n)$$

Related task: probability of an upcoming word:

$$P(w5|w1,w2,w3,w4)$$

A model that computes either of these:

$$P(W) \quad \text{or} \quad P(wn|w1,w2 \ldots wn\text{-}1)$$

is called a language model.

- Models that assign probabilities to sequences of words are called language models
- N-gram is a simple model that assigns probabilities to sentences and sequences of words.

Perplexity

The best language model is one that best predicts an unseen test set

- Gives the highest P(sentence)

Perplexity is the inverse probability of the test set, normalized by the number of words:

$$PP(W) = P(w_1 w_2 \ldots w_N)^{-\frac{1}{N}}$$

$$= \sqrt[N]{\frac{1}{P(w_1 w_2 \ldots w_N)}}$$

Chain rule:

$$\mathrm{PP}(W) = \sqrt[N]{\prod_{i=1}^{N} \frac{1}{P(w_i | w_1 \ldots w_{i-1})}}$$

For bigrams:

$$\mathrm{PP}(W) = \sqrt[N]{\prod_{i=1}^{N} \frac{1}{P(w_i | w_{i-1})}}$$

Minimizing perplexity is the same as maximizing probability

Backoff and Interpolation

Sometimes it helps to use **less** context

- Condition on less context for contexts you haven't learned much about

Backoff:

- use trigram if you have good evidence,
- otherwise bigram, otherwise unigram

Interpolation:

- mix unigram, bigram, trigram

Interpolation works better

Neural Network Unit

This is not in your brain

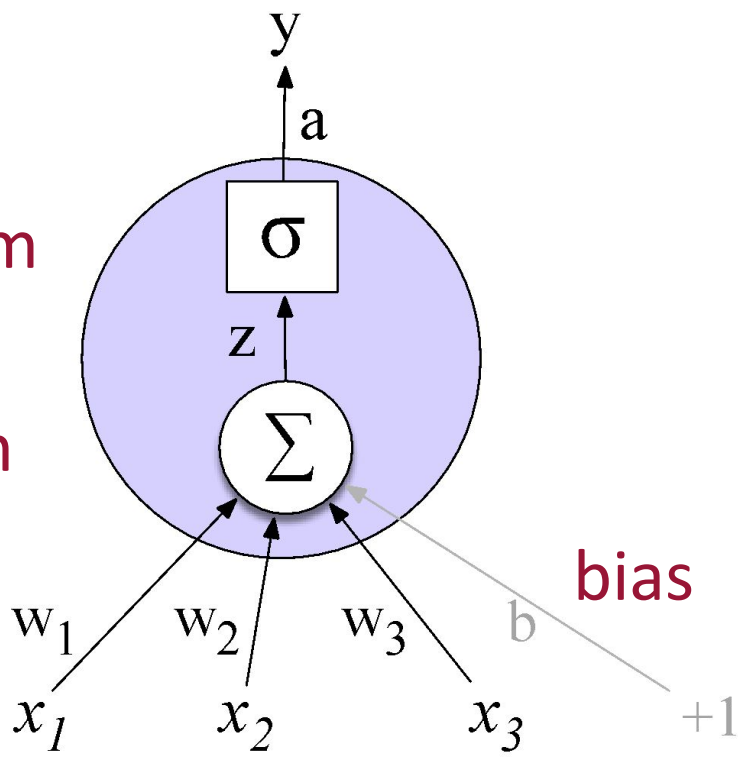
Output value

Non-linear transform

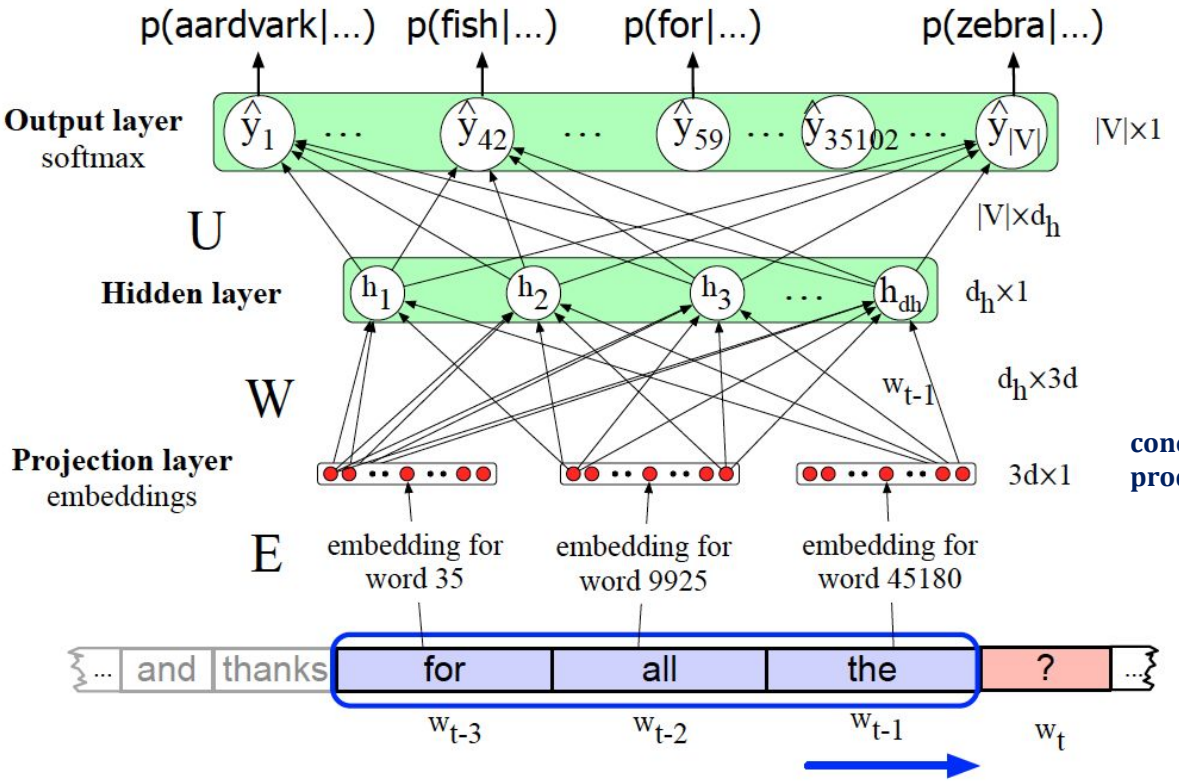
Weighted sum

Weights

Input layer



Neural Language Model

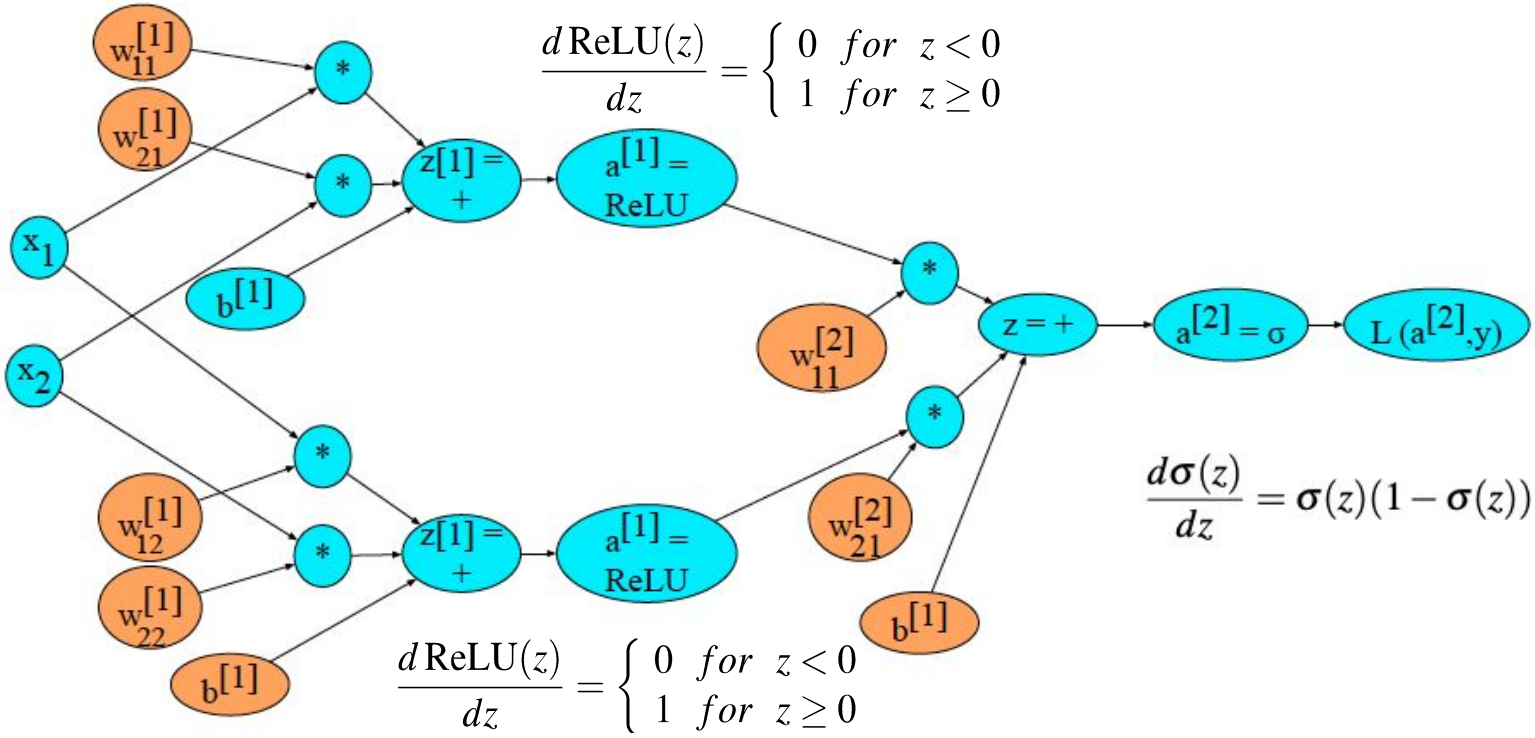


Equations:

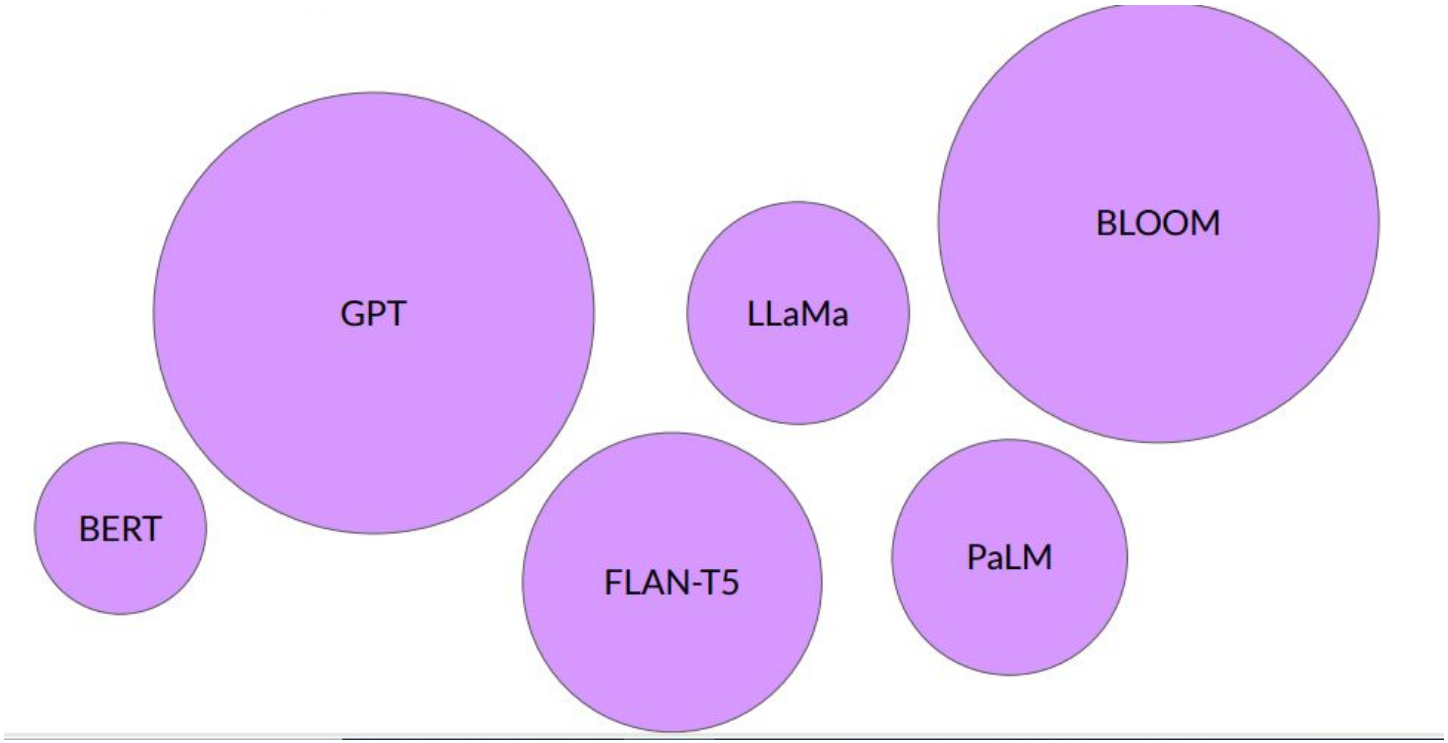
$$
\begin{aligned}
\mathbf{e} &= [\mathbf{E}x_{t-3}; \mathbf{E}x_{t-2}; \mathbf{E}x_{t-1}] \\
\mathbf{h} &= \sigma(\mathbf{W}\mathbf{e} + \mathbf{b}) \\
\mathbf{z} &= \mathbf{U}\mathbf{h} \\
\hat{\mathbf{y}} &= \text{softmax}(\mathbf{z})
\end{aligned}
$$

concatenate 3 embeddings for the 3 context words to produce the embedding layer e

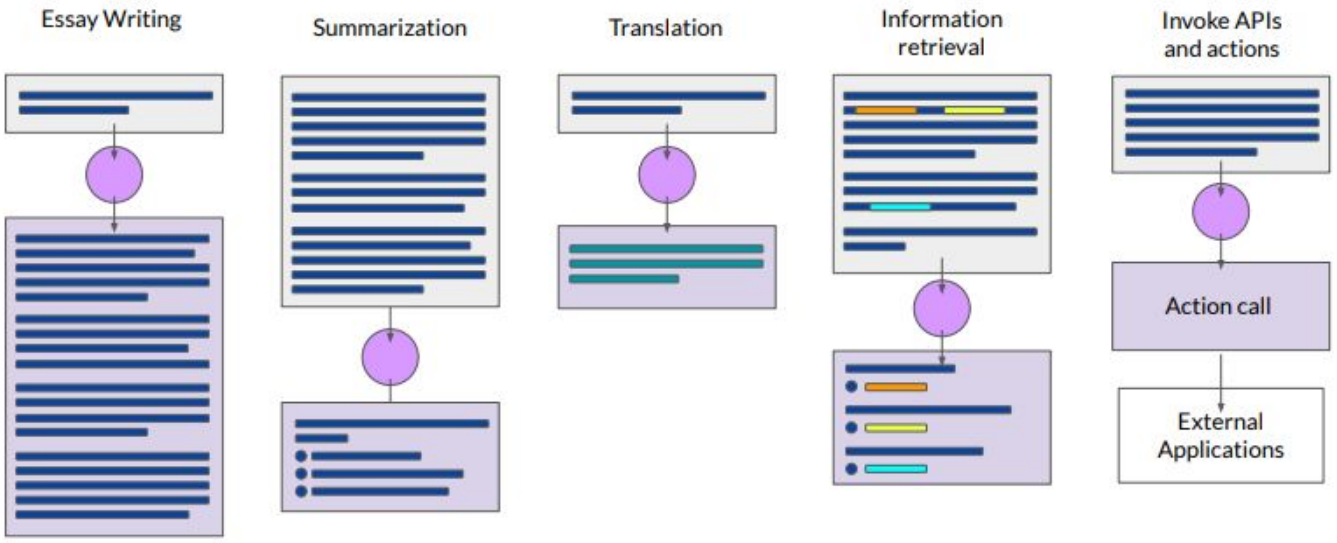
Backward differentiation on a 2-layer network



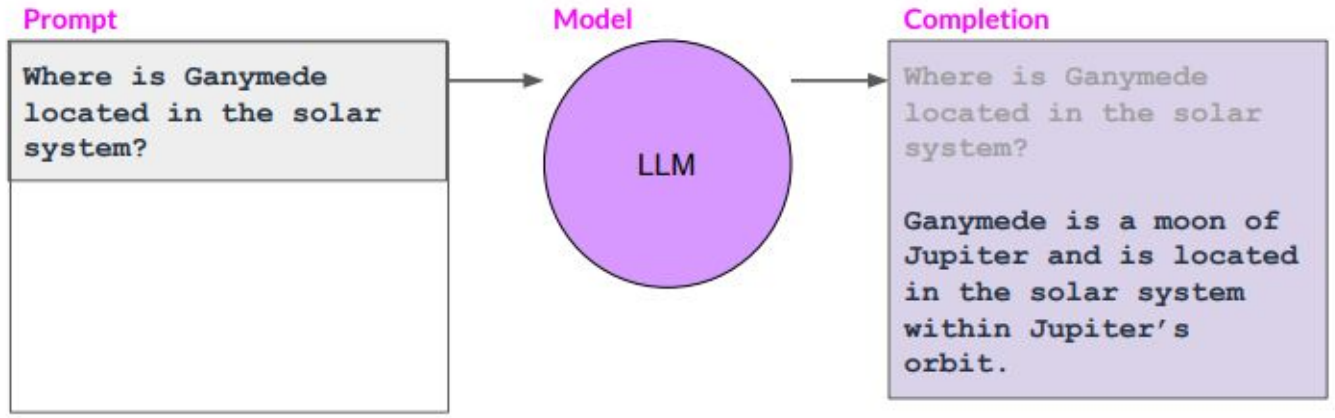
Large Language Models



LLM Use Cases

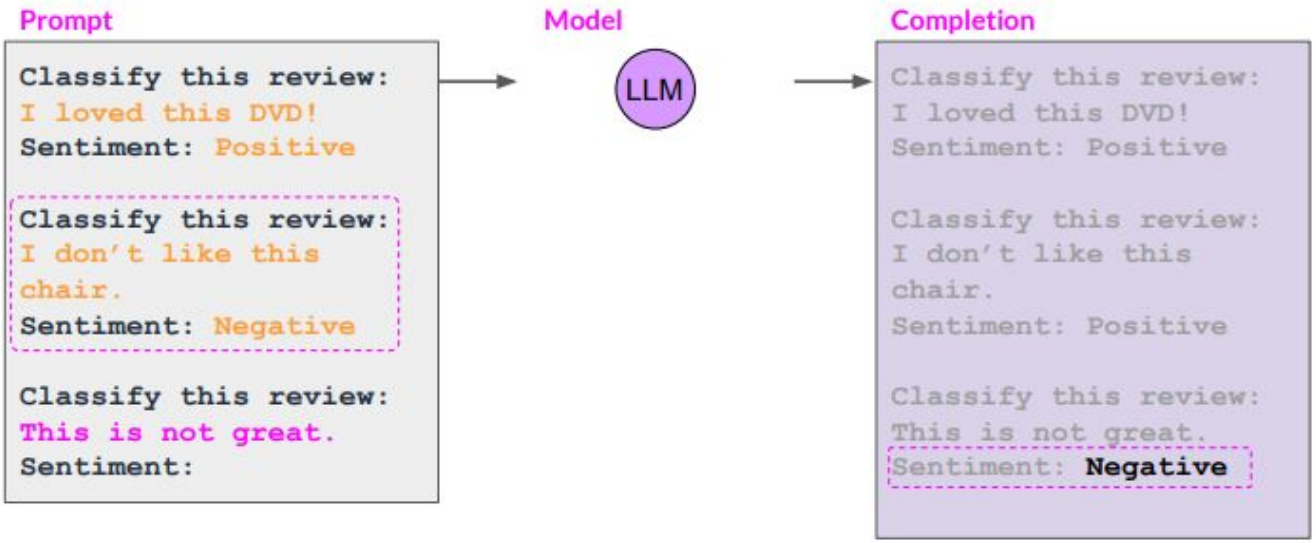


Prompting and Prompt Engineering



Context window: typically a few thousand words

In context learning and Few shot inference



Transfer Learning

- Using a pre-trained model as a starting point for a new task or domain.
- Leverage the knowledge acquired by the pre-trained model on a large dataset and apply it to a related task with a smaller dataset.
- We can benefit from the general features and patterns learned by the pre-trained model, saving time and computational resources.
- Transfer learning involves freezing the pre-trained model's weights and only training the new layers

Ex: image classification, knowledge gained while learning to recognize cars could be applied when trying to recognize trucks

Two classes of words: Open vs. Closed

Closed class words

- Relatively fixed membership
- Usually **function** words: short, frequent words with grammatical function
  - determiners: ***a, an, the***
  - pronouns: ***she, he, I***
  - prepositions: ***on, under, over, near, by, …***

Open class words

- Usually **content** words: Nouns, Verbs, Adjectives, Adverbs
  - Plus interjections: **oh, ouch, uh-huh, yes, hello**
- New nouns and verbs like *iPhone* or *to fax*

"Universal Dependencies" Tagset

	Tag	Description	Example
Open Class	ADJ	Adjective: noun modifiers describing properties	*red, young, awesome*
	ADV	Adverb: verb modifiers of time, place, manner	*very, slowly, home, yesterday*
	NOUN	words for persons, places, things, etc.	*algorithm, cat, mango, beauty*
	VERB	words for actions and processes	*draw, provide, go*
	PROPN	Proper noun: name of a person, organization, place, etc..	*Regina, IBM, Colorado*
	INTJ	Interjection: exclamation, greeting, yes/no response, etc.	*oh, um, yes, hello*
Closed Class Words	ADP	Adposition (Preposition/Postposition): marks a noun's spacial, temporal, or other relation	*in, on, by under*
	AUX	Auxiliary: helping verb marking tense, aspect, mood, etc.,	*can, may, should, are*
	CCONJ	Coordinating Conjunction: joins two phrases/clauses	*and, or, but*
	DET	Determiner: marks noun phrase properties	*a, an, the, this*
	NUM	Numeral	*one, two, first, second*
	PART	Particle: a preposition-like form used together with a verb	*up, down, on, off, in, out, at, by*
	PRON	Pronoun: a shorthand for referring to an entity or event	*she, who, I, others*
	SCONJ	Subordinating Conjunction: joins a main clause with a subordinate clause such as a sentential complement	*that, which*
Other	PUNCT	Punctuation	; , ()
	SYM	Symbols like $ or emoji	$, %
	X	Other	asdf, qwfg

Nivre et al. 2016

Named Entities

Why NER?

Sentiment analysis: consumer's sentiment toward a particular company or person?

Question Answering: answer questions about an entity?

Information Extraction: Extracting facts about entities from text.

Named entity, in its core usage, means anything that can be referred to with a proper name. Most common 4 tags:

- PER (Person): "Marie Curie"
- LOC (Location): "New York City"
- ORG (Organization): "Stanford University"
- GPE (Geo-Political Entity): "Boulder, Colorado"

– Often multi-word phrases

– But the term is also extended to things that aren't entities:

  - dates, times, prices

BIO Tagging

[PER Jane Villanueva] of [ORG United] , a unit of [ORG United Airlines Holding] , said the fare applies to the [LOC Chicago] route.

Words	BIO Label
Jane	B-PER
Villanueva	I-PER
of	O
United	B-ORG
Airlines	I-ORG
Holding	I-ORG
discussed	O
the	O
Chicago	B-LOC
route	O
.	O

B: token that *begins* a span

I: tokens *inside* a span

O: tokens outside of any span

Now we have one tag per token!!!

Rule Based Tagging - Brill tagger

- The tagger works by automatically recognizing and remedying its weaknesses, thereby incrementally improving its performance.
- The tagger initially tags by assigning each word its most likely tag, estimated by examining a large tagged corpus, without regard to context.
- Tagger uses 2 initial procedures
- One procedure is provided with information that words that were not in the training corpus and are capitalized tend to be proper nouns
- To tag words not seen in the training corpus by assigning such words the tag most common for words ending in the same three letters. For example, blahblahous would be tagged as an adjective

A Simple Rule-Based Part of Speech Tagger, Eric Brill Department of Computer Science, University of Pennsylvania

Hidden Markov Model

- A Markov chain is useful when we need to compute a probability for a sequence of observable events.
    - We don't observe part-of-speech tags in a text. Rather, we see words, and must infer the tags from the word sequence.
    - We call the tags hidden because they are not observed
- A hidden Markov model (HMM) includes both observed events model (like words that we see in the input) and hidden events (like part-of-speech tags) that we think of as causal factors in our probabilistic model.

Hidden Markov Model

$Q = q_1 q_2 \ldots q_N$	a set of N **states**
$A = a_{11} \ldots a_{ij} \ldots a_{NN}$	a **transition probability matrix** A, each a_{ij} representing the probability of moving from state i to state j, s.t. $\sum_{j=1}^{N} a_{ij} = 1 \quad \forall i$
$O = o_1 o_2 \ldots o_T$	a sequence of T **observations**, each one drawn from a vocabulary $V = v_1, v_2, ..., v_V$
$B = b_i(o_t)$	a sequence of **observation likelihoods**, also called **emission probabilities**, each expressing the probability of an observation o_t being generated from a state q_i
$\pi = \pi_1, \pi_2, ..., \pi_N$	an **initial probability distribution** over states. π_i is the probability that the Markov chain will start in state i. Some states j may have $\pi_j = 0$, meaning that they cannot be initial states. Also, $\sum_{i=1}^{n} \pi_i = 1$

A matrix contains the tag transition probabilities which represent the probability of a tag occurring given the previous tag

$$P(t_i|t_{i-1}) = \frac{C(t_{i-1}, t_i)}{C(t_{i-1})}$$

The MLE of the emission probability is

$$P(w_i|t_i) = \frac{C(t_i, w_i)}{C(t_i)}$$

HMM Part-of-Speech Tagging

Plugging the simplifying assumptions from previous slide, we get the following equation for the most probable tag sequence from a bigram tagger

$$\hat{t}_{1:n} = \underset{t_1 \ldots t_n}{\operatorname{argmax}} P(t_1 \ldots t_n | w_1 \ldots w_n) \approx \underset{t_1 \ldots t_n}{\operatorname{argmax}} \prod_{i=1}^{n} \overbrace{P(w_i|t_i)}^{\text{emission}} \overbrace{P(t_i|t_{i-1})}^{\text{transition}}$$

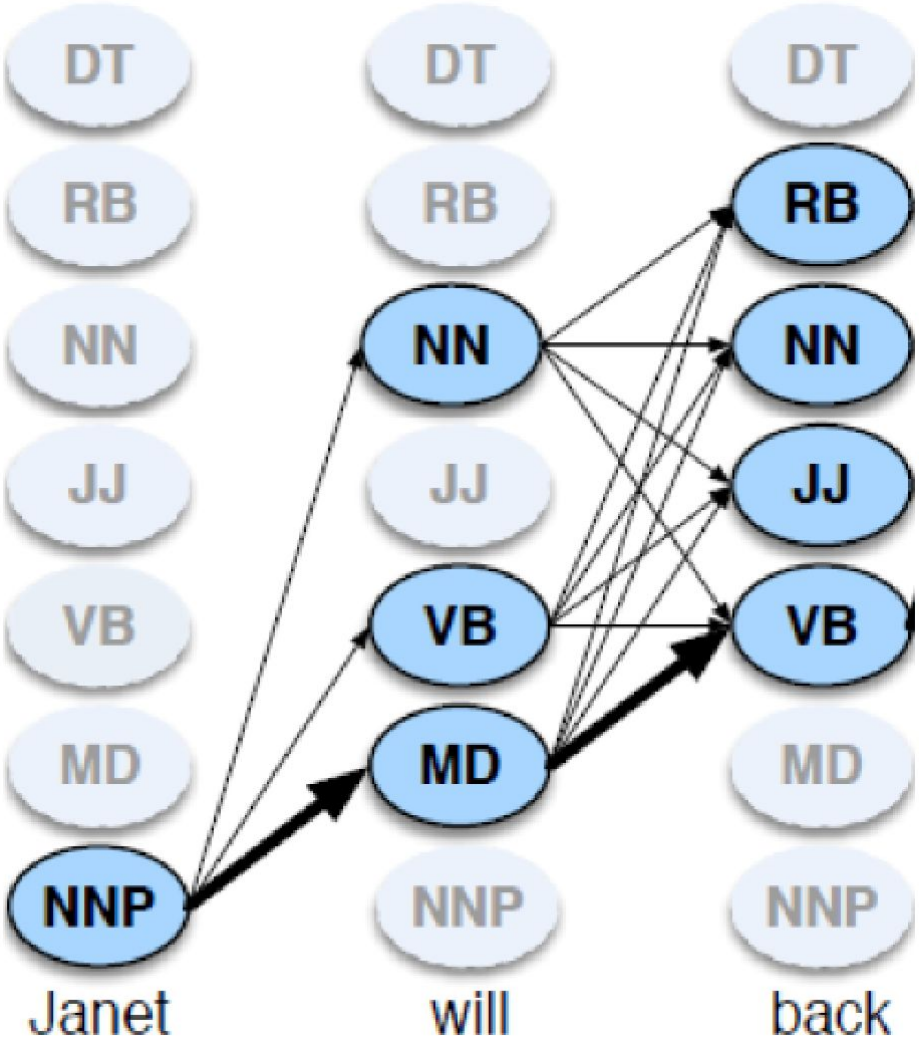
The two parts correspond to the B emission probability and A transition probability.

The Viterbi Algorithm

function VITERBI(*observations* of len *T*,*state-graph* of len *N*) **returns** *best-path*, *path-prob*

create a path probability matrix *viterbi*[*N*,*T*]
for each state *s* **from** 1 **to** *N* **do** ; initialization step
  $viterbi[s,1] \leftarrow \pi_s * b_s(o_1)$
  $backpointer[s,1] \leftarrow 0$
for each time step *t* **from** 2 **to** *T* **do** ; recursion step
  for each state *s* **from** 1 **to** *N* **do**
    $viterbi[s,t] \leftarrow \max_{s'=1}^{N} viterbi[s',t-1] * a_{s',s} * b_s(o_t)$
    $backpointer[s,t] \leftarrow \operatorname{argmax}_{s'=1}^{N} viterbi[s',t-1] * a_{s',s} * b_s(o_t)$
$bestpathprob \leftarrow \max_{s=1}^{N} viterbi[s,T]$; termination step
$bestpathpointer \leftarrow \operatorname{argmax}_{s=1}^{N} viterbi[s,T]$; termination step
bestpath ← the path starting at state *bestpathpointer*, that follows backpointer[] to states back in time
return *bestpath*, *bestpathprob*

Viterbi Algorithm for POS Tagging Example



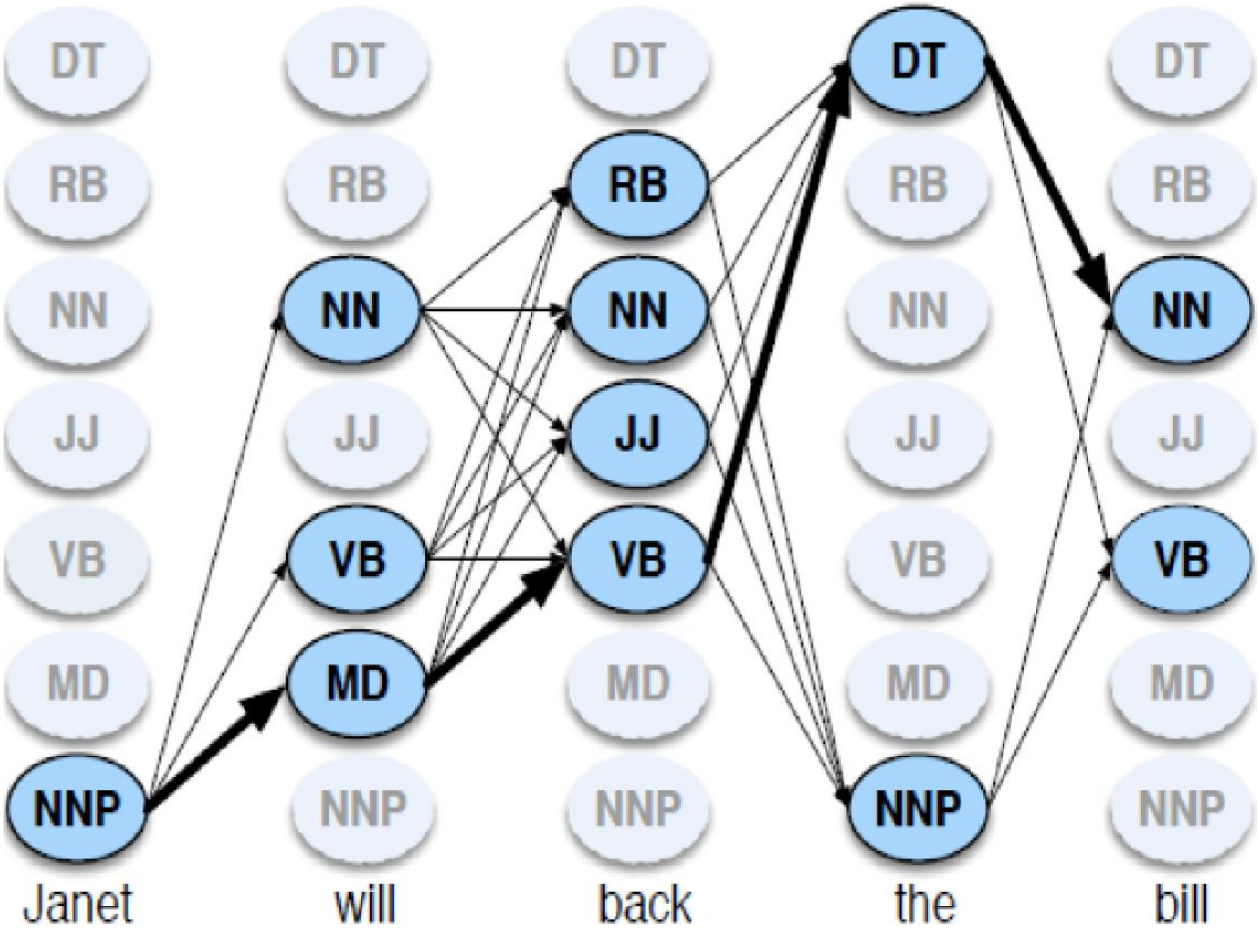
Viterbi[RB,back]=max({Viterbi[NN,will]*P(RB|NN)*P(back|RB),
Viterbi[VB,will]*P(RB|VB)*P(back|RB),
Viterbi[MD,will]*P(RB|MD)*P(back|RB)})

Viterbi[NN,back]=max({Viterbi[NN,will]*P(NN|NN)*P(back|NN),
Viterbi[VB,will]*P(NN|VB)*P(back|NN),
Viterbi[MD,will]*P(NN|MD)*P(back|NN)})

Viterbi[JJ,back]=max({Viterbi[NN,will]*P(JJ|NN)*P(back|JJ),
Viterbi[VB,will]*P(JJ|VB)*P(back|JJ),
Viterbi[MD,will]*P(JJ|MD)*P(back|JJ)})

Viterbi[VB,back]=max({Viterbi[NN,will]*P(VB|NN)*P(back|VB),
Viterbi[VB,will]*P(VB|VB)*P(back|VB),
Viterbi[MD,will]*P(VB|MD)*P(back|VB)})

Viterbi Algorithm for POS Tagging Example



Viterbi[NN,bill]=
max({Viterbi[DT,the]*P(NN|DT)*P(bill|NN),
Viterbi[NNP,the]*P(NN|NNP)*P(bill|NN)})

Viterbi[VB,bill]=
max({Viterbi[DT,the]*P(VB|DT)*P(bill|VB),
Viterbi[NNP,the]*P(VB|NNP)*P(bill|VB)})

Conditional Random Fields (CRFs)

CRF is discriminative sequence model based on log-linear models: on lines of logistic regression for single observation

Say, we have a sequence of input words.

$$X = x_1^n = x_1 \ldots x_n$$

and want to compute a sequence of output tags

$$Y = y_1^n = y_1 \ldots yn$$

In an HMM to compute the best tag sequence

$$\begin{aligned}
\hat{Y} &= \operatorname*{argmax}_Y p(Y|X) \\
&= \operatorname*{argmax}_Y p(X|Y)p(Y) \\
&= \operatorname*{argmax}_Y \prod_i p(x_i|y_i) \prod_i p(y_i|y_{i-1})
\end{aligned}$$

Conditional Random Fields (CRFs)

In a CRF, we compute the posterior p(Y|X) directly, training the CRF to discriminate among the possible tag sequences

$$\hat{Y} = \underset{Y \in \mathcal{Y}}{\operatorname{argmax}} P(Y|X)$$

- The CRF does not compute a probability for each tag at each time step.
- At each time step the CRF computes log-linear functions over a set of relevant features, and these local features are aggregated and normalized to produce a global probability for the whole sequence

Here are some example features

$$\mathbb{1}\{x_i = the,\ y_i = \text{DET}\}$$
$$\mathbb{1}\{y_i = \text{PROPN},\ x_{i+1} = Street,\ y_{i-1} = \text{NUM}\}$$
$$\mathbb{1}\{y_i = \text{VERB},\ y_{i-1} = \text{AUX}\}$$

The reason to use a discriminative sequence model is that it's easier to incorporate a lot of features

For simplicity, assume all CRF features take on the value 1 or 0

For the example Janet/NNP will/MD back/VB the/DT bill/NN, when x_i is the word back, the following features would be generated with value of 1:

$$f_{3743}: y_i = \text{VB and } x_i = \text{back}$$
$$f_{156}: y_i = \text{VB and } y_{i-1} = \text{MD}$$
$$f_{99732}: y_i = \text{VB and } x_{i-1} = \text{will and } x_{i+2} = \text{bill}$$

More on HMM

Hidden Markov models should be characterized by three fundamental problems

Problem 1 (Likelihood):	Given an HMM $\lambda = (A, B)$ and an observation sequence O, determine the likelihood $P(O\|\lambda)$.
Problem 2 (Decoding):	Given an observation sequence O and an HMM $\lambda = (A, B)$, discover the best hidden state sequence Q.
Problem 3 (Learning):	Given an observation sequence O and the set of states in the HMM, learn the HMM parameters A and B.

What we discussed is using HMM for problem 2.

Thank You

References

	Author(s), Title, Edition, Publishing House
T1	Speech and Language processing: An introduction to Natural Language Processing, Computational Linguistics and speech Recognition by Daniel Jurafsky and James H. Martin[3rd edition]
T2	Foundations of statistical Natural language processing by Christopher D.Manning and Hinrich schutze